Baze de date
Laborator 3

Interogări multi-relaţie. Operaţia de join. Operatori pe mulţimi.

I. [Join]

Join-ul este operaţia de regăsire a datelor din două sau mai multe tabele, pe baza valorilor comune ale unor coloane. De obicei, aceste coloane reprezintă **cheia primară**, respectiv **cheia externă** a tabelelor.

Pentru a realiza un **JOIN** între **n tabele**, va fi nevoie de cel puţin **n – 1 condiţii de join.**

Tipuri de JOIN :

- ***Inner join (equijoin, join simplu)*** – corespunde situaţiei în care valorile de pe coloanele ce apar în condiţia de *join* trebuie să fie egale.

- ***Nonequijoin –*** condiţia de *join* conţine alţi operatori decât operatorul de egalitate.

  Exemplu Nonequijoin:

  SELECT last_name, salary, grade_level, lowest_sal, highest_sal
  FROM employees, job_grades
  WHERE salary BETWEEN lowest_sal AND highest_sal;

- ***Left | Right Outer join*** – un *outer join* este utilizat pentru a obţine în rezultat şi înregistrările care nu satisfac condiţia de *JOIN*.

  Operatorul pentru ***outer join*** este semnul plus inclus între paranteze **(+)**, care se plasează în acea parte a condiţiei de *join* care este **deficitară în informaţie**. Efectul acestui operator este de a uni liniile tabelului care nu este deficitar în informaţie, cărora nu le corespunde nicio linie în celălalt tabel, cu o linie cu valori *NULL*. Operatorul (+) poate fi plasat în orice parte a condiţiei de *join*, dar **nu în ambele părţi**.

Obs: O condiţie care presupune un *outer join* nu poate utiliza operatorul *IN* şi nu poate fi legată de altă condiţie prin operatorul *OR*.

- ***Full outer join*** – left outer join + right outer join

- ***Self join*** – join-ul unui tabel cu el însuşi. În ce situaţie concretă (relativ la modelul nostru) apărea această operaţie?

CEZARA BENEGUI

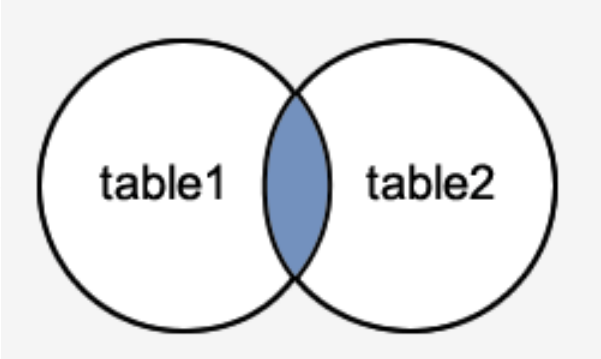
Într-o instrucţiune *SELECT* care uneşte tabele prin **operaţia de *join***, se recomandă ca numele coloanelor să fie precedate de numele sau alias-urile tabelelor pentru claritate şi pentru îmbunătăţirea timpului de acces la baza de date. Dacă acelaşi nume de coloană apare în mai mult de două tabele, atunci numele coloanei se prefixează **obligatoriu** cu numele sau *alias*-ul tabelului corespunzător.

Exemple practice și explicații detaliate:

1. **Inner join *(equijoin, join simplu)*** – corespunde situaţiei în care valorile de pe coloanele ce apar în condiţia de *join* trebuie să fie egale. (**returnează toate rândurile din mai multe tabele – din tabelele specificate în condiția de join – în care condiția de join este îndeplinită**).

   Structură*:*

```
SELECT columns
FROM table1 INNER JOIN table2 ON (table1.column = table2.column);
```



După cum se observă, **INNER JOIN** returnează rândurile (rows) pentru care condiția de join este îndeplinită.

CEZARA BENEGUI

EXEMPLU IMPLEMENTARE: Să se afişeze **codul angajaților** şi **numele departamentului** pentru toţi angajaţii care lucrează în departamente.

I. Condiția de JOIN este scrisă în clauza WHERE a instrucțiunii SELECT

SELECT employee_id, department_name
FROM employees e, departments d
WHERE e.department_id = d.department_id;

II. Condiția de JOIN este scrisă în FROM – Standardul SQL3

- **Utilizând ON -** efectuează un *join* pe baza condiţiei exprimate în clauza *ON*. Această clauză permite specificarea separată a condiţiilor de *join*

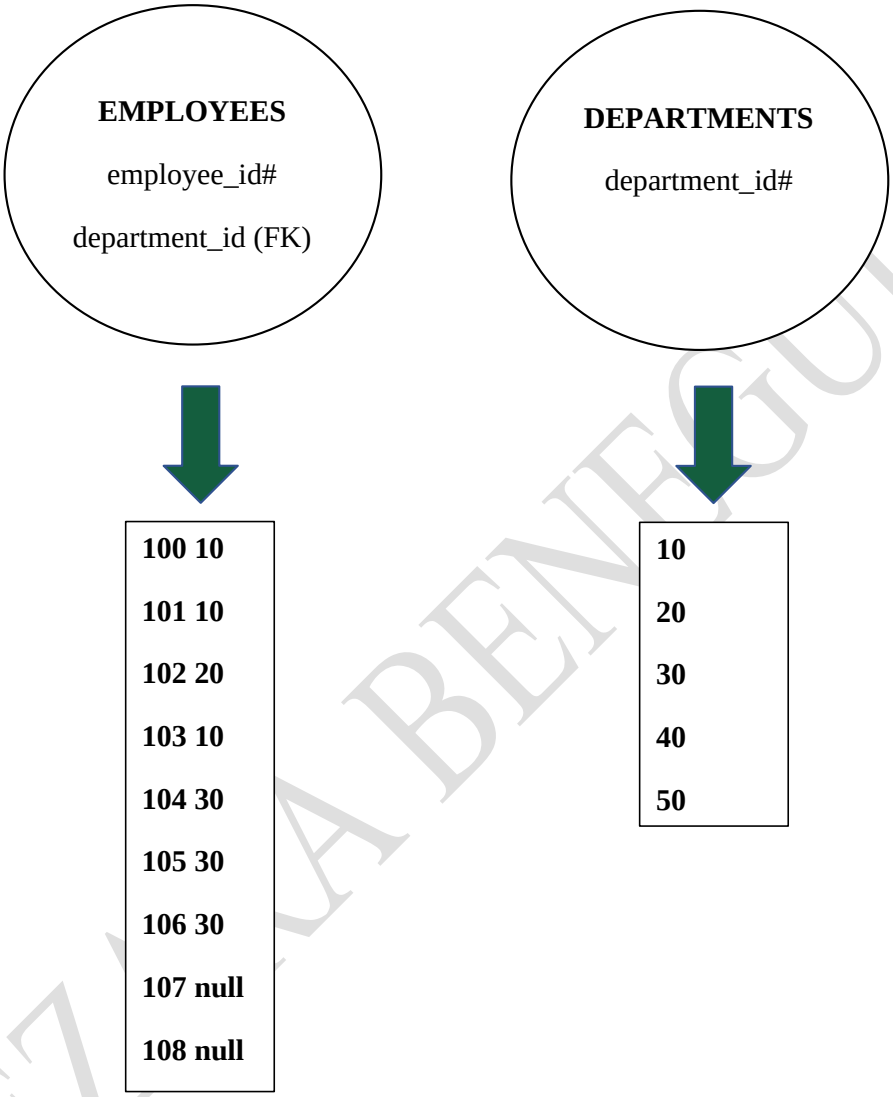
SELECT employee_id, department_name
FROM employees e JOIN departments d ON (e.department_id = d.department_id);

- **Utilizând USING –** efectuează un ***equijoin*** pe baza coloanei cu numele specificat în sintaxă. Această clauză este utilă dacă există coloane având acelaşi nume. Coloanele referite în clauza *USING* trebuie să nu conţină calificatori (să nu fie precedate de nume de tabele sau *alias*-uri) în nicio apariţie a lor în instrucţiunea *SQL*.

SELECT employee_id,department_name
FROM employees e JOIN departments d USING (department_id);

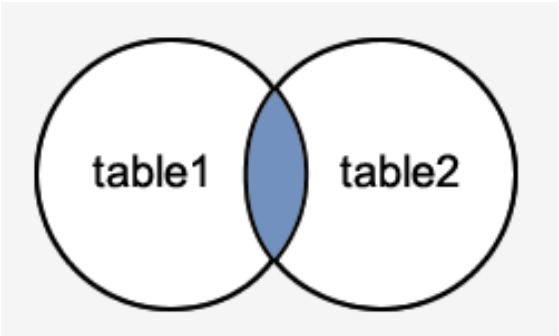
CEZARA BENEGUI

Pentru exemplificare vom lua câteva exemple:



CEZARA BENEGUI

EMPLOYEES **DEPARTMENTS**



100	**10**
101	**10**
102	**20**
103	**10**
104	**30**
105	**30**
106	**30**

INNER JOIN – au fost afișați toți ANGAJAȚII CARE LUCREAZĂ în departamente!

Obs: Am realizat operaţia de join între tabelele ***employees*** şi ***departments***, pe baza coloanei comune ***department_id***.

Observaţi utilizarea *alias*-urilor. Numele sau *alias*-urile tabelelor sunt obligatorii în dreptul coloanelor care au acelaşi nume în mai multe tabele. Altfel, nu sunt necesare dar este recomandată utilizarea lor pentru o mai bună claritate a cererii.

Ce se întâmplă dacă eliminăm condiţia de *join*?

CEZARA BENEGUI

2. **Nonequijoin –** condiţia de **JOIN** conţine alţi operatori decât operatorul de egalitate.

   Exemplu Nonequijoin:

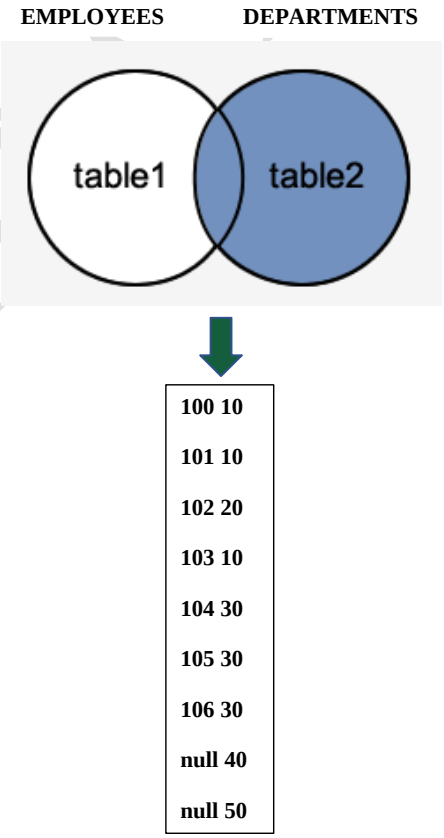
   SELECT last_name, salary, grade_level, lowest_sal, highest_sal
   FROM employees, job_grades
   WHERE salary BETWEEN lowest_sal AND highest_sal;

3. **LEFT | RIGHT | FULL OUTER JOIN** – un **OUTER JOIN** este utilizat pentru a obţine în rezultat şi înregistrările care nu satisfac condiţia de **JOIN**.

   LEFT*, *RIGHT* şi *FULL OUTER JOIN tabel_2 ON (*tabel_1.nume_coloană = tabel_2.nume_coloană*) efectuează *outer join* la stânga, dreapta, respectiv în ambele părţi pe baza condiţiei exprimate în clauza *ON*.

   Un *join* care returnează rezultatele unui *inner join*, dar şi cele ale *outer join*-urilor la stânga şi la dreapta se numeşte *full outer join*.

EXEMPLU IMPLEMENTARE: Să se afişeze **codul angajaților** şi **numele departamentului** pentru toţi angajaţii. Să se afișeze toate departamentele, chiar dacă au sau nu angajați (se vor afișa atât departamentele în care lucrează angajați, cât și departamentele care nu au angajați).



CEZARA BENEGUI

I. Condiția de JOIN este scrisă în clauza WHERE a instrucțiunii SELECT

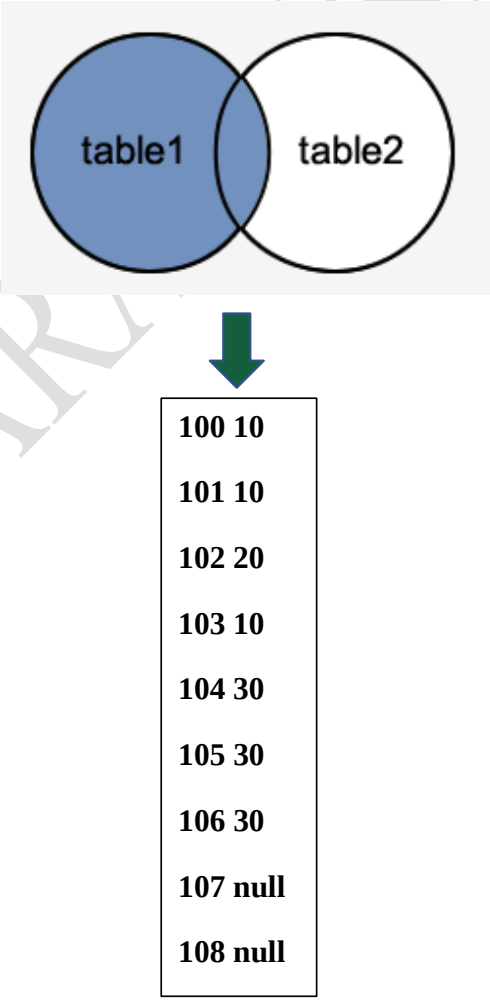
SELECT employee_id, department_name
FROM employees e, departments d
WHERE e.department_id(+) = d.department_id;

II. Condiția de JOIN este scrisă în FROM – Standardul SQL3

Utilizând ON:

SELECT employee_id, department_name
FROM employees e ***RIGHT JOIN*** *departments d ON (e.department_id = d.department_id);*

EXEMPLU IMPLEMENTARE: Să se afişeze **codul angajaților** şi **numele departamentului** pentru toţi angajaţii. Să se afișeze toți angajații, chiar dacă au sau nu departament (se vor afișa atât angajații care lucrează într-un departament, cât și angajații care nu au departament în acest moment).



CEZARA BENEGUI

I. Condiția de JOIN este scrisă în clauza WHERE a instrucțiunii SELECT

SELECT employee_id, department_name
FROM employees e, departments d
WHERE e.department_id = d.department_id(+);

II. Condiția de JOIN este scrisă în FROM – Standardul SQL3

Utilizând ON:

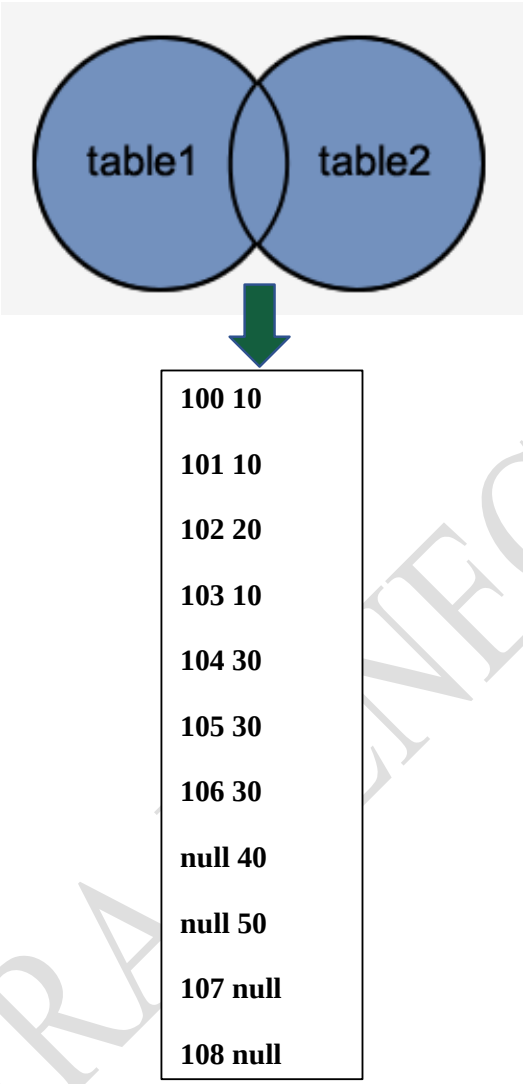
SELECT employee_id, department_name
FROM employees e ***LEFT JOIN*** *departments d ON (e.department_id = d.department_id);*

EXEMPLU IMPLEMENTARE: Să se afişeze **codul angajaților** şi **numele departamentului** pentru toţi angajaţii. Să se afișeze toți angajații, chiar dacă au sau nu departament, dar și toate departamentele chiar dacă au sau nu angajați.

În acest caz, se vor afișa:

- angajații care lucrează în departamente (sau departamentele care au angajați) – intersecția
- la care se adaugă departamentele care nu au angajați
- și angajații care nu au departament

CEZARA BENEGUI



CEZARA BENEGUI

I. Condiția de JOIN este scrisă în clauza WHERE a instrucțiunii SELECT

Operatorul (+) poate fi plasat în orice parte a condiţiei de *join*, dar **nu în ambele părţi**.

II. Condiția de JOIN este scrisă în FROM – Standardul SQL3

Utilizând ON:

SELECT employee_id, department_name
FROM employees e **FULL JOIN** departments d ON (e.department_id = d.department_id);

Join introdus în standardul SQL3 (SQL:1999)*:*

Pentru **JOIN**, sistemul *Oracle* oferă şi o sintaxă specifică, în conformitate cu standardul *SQL3* (*SQL*:*1999*). Această sintaxă nu aduce beneficii, în privinţa performanţei, faţă de *join*-urile care folosesc sintaxa utilizată anterior. Tipurile de *join* conforme cu *SQL3* sunt definite prin cuvintele cheie *CROSS JOIN* (pentru produs cartezian), *NATURAL JOIN*, *FULL OUTER JOIN*, clauzele *USING* şi *ON*.

Sintaxa corespunzătoare standardului *SQL3* este următoarea:

SELECT *tabel_1.nume_coloană*, *tabel_2.nume_coloană*
FROM *tabel_1*
[***CROSS JOIN*** *tabel_2*]
| [***NATURAL JOIN*** *tabel_2*]
| [***JOIN*** *tabel_2* ***USING*** (*nume_coloană*)]
| [***JOIN*** *tabel_2* ***ON*** (*conditie*)]
| [***LEFT*** | ***RIGHT*** | ***FULL OUTER JOIN*** *tabel_2*
ON (*tabel_1.nume_coloană = tabel_2.nume_coloană*)];

- ***NATURAL JOIN*** presupune existenţa unor coloane având acelaşi nume în ambele tabele. Clauza determină selectarea liniilor din cele două tabele, care au valori egale în aceste coloane. Dacă tipurile de date ale coloanelor cu nume identice sunt diferite, va fi returnată o eroare.

  Coloanele având acelaşi nume în cele două tabele trebuie să nu fie precedate de numele sau *alias*-ul tabelului corespunzător.

  Clauzele *NATURAL JOIN* şi *USING* nu pot coexista în aceeaşi instrucţiune *SQL*.

Exemplu:

```
SELECT last_name, job_id, job_title
FROM employees
NATURAL JOIN jobs;
```

```
SELECT last_name, e.job_id, job_title
FROM employees e, jobs j
WHERE e.job_id = j.job_id;
```

CEZARA BENEGUI

II. [Exerciţii - join]

1. Să se listeze **codurile** și **denumirile job-urilor** care există în departamentul 30.

2. Să se afişeze **numele angajatului**, **numele departamentului** şi **id-ul locației** pentru toţi angajaţii care câştigă comision.

SELECT ____, _____, _____
FROM ____, _____
WHERE_____ AND commission_pct _____ ;

3. Să se afişeze **numele angajaților**, **titlul job-ului** şi **denumirea departamentului** pentru toţi angajaţii care lucrează în Oxford (coloana - city).

4. Să se afişeze **codul angajatului** şi **numele** acestuia, împreună cu **numele** şi **codul şefului** său direct. Se vor eticheta coloanele Cod Angajat, Nume Angajat, Cod Manager, Nume Manager.

SELECT ang.employee_id "Cod Angajat", ang.last_name "Nume Angajat",
mgr.employee_id "Cod Manager", mgr.last_name "Nume Manager"
FROM employees ang JOIN employees mgr
ON (ang.manager_id = mgr.employee_id);

Obs: Am realizat operaţia de *self*-join (inner join al tabelului cu el însuşi).

5. Să se modifice cererea anterioară pentru a afişa toţi salariaţii, inclusiv cei care nu au şef.

6. Scrieţi o cerere care afişează **numele angajatului, codul departamentului** în care acesta lucrează şi **numele colegilor** săi de departament. Se vor eticheta coloanele corespunzător.

7. Creaţi o cerere prin care să se afişeze **numele angajatilor**, **codul job-ului**, **titlul job-ului**, **numele departamentului** şi **salariul** angajaţilor. Se vor include și angajații al căror departament nu este cunoscut.

CEZARA BENEGUI

- **JOIN în WHERE**

```
SELECT last_name, j.job_id, job_title, department_name, salary
FROM employees e, departments d, jobs j
WHERE e.department_id = d.department_id (+)
            AND  j.job_id = e.job_id;
```

- **JOIN în FROM – să se implementeze și această variantă (Standardul SQL3)**

```
SELECT last_name, j.job_id, job_title, department_name, salary
FROM ____;
```

8. Să se afişeze **numele** şi **data angajării** pentru salariaţii care au fost angajaţi după salariatul cu numele (last_name) *Gates*.

9. Scrieți o cerere pentru a afișa **numele salariatului**, **luna** (în litere), **anul angajării** și valoarea comisionului pentru toți salariații din același departament cu Gates (last_name este Gates) – se verifică numele scris cu prima literă mare și restul literelor mici, al căror nume conţine litera “a”. Se va exclude Gates. Se vor utiliza aliasuri pentru numele coloanelor din output. În cazul în care un angajat nu câștigă comision, se va scrie în output, pe coloana respectivă, mesajul “Nu câștigă comision”. Rezultatul se va ordona alfabetic după numele salariaților. Output-ul este următorul:

CEZARA BENEGUI

	Nume Angajat	Luna si an	Departament Angajat	Nume Gates	Departament Gates	Comision angajat
1	Atkinson	october -1997	50	Gates	50	Nu castiga comision
2	Cabrio	february -1999	50	Gates	50	Nu castiga comision
3	Davies	january -1997	50	Gates	50	Nu castiga comision
4	Fleaur	february -1998	50	Gates	50	Nu castiga comision
5	Grant	january -2000	50	Gates	50	Nu castiga comision
6	Kaufling	may -1995	50	Gates	50	Nu castiga comision
7	Ladwig	july -1995	50	Gates	50	Nu castiga comision
8	Landry	january -1999	50	Gates	50	Nu castiga comision
9	Mallin	june -1996	50	Gates	50	Nu castiga comision
10	Markle	march -2000	50	Gates	50	Nu castiga comision
11	Marlow	february -1997	50	Gates	50	Nu castiga comision
12	Matos	march -1998	50	Gates	50	Nu castiga comision
13	McCain	july -1998	50	Gates	50	Nu castiga comision
14	Naver	july -1997	50	Gates	50	Nu castiga comision
15	Patel	april -1998	50	Gates	50	Nu castiga comision
16	Philtanker	february -2000	50	Gates	50	Nu castiga comision
17	Rajs	october -1995	50	Gates	50	Nu castiga comision
18	Sarchand	january -1996	50	Gates	50	Nu castiga comision
19	Sullivan	june -1999	50	Gates	50	Nu castiga comision
20	Taylor	january -1998	50	Gates	50	Nu castiga comision
21	Vargas	july -1998	50	Gates	50	Nu castiga comision
22	Vollman	october -1997	50	Gates	50	Nu castiga comision
23	Walsh	april -1998	50	Gates	50	Nu castiga comision

10. Să se afișeze numele, salariul, titlul job-ului, oraşul şi ţara în care lucrează angajații conduși direct de King.

11. Să se afișeze codul departamentului, numele departamentului, numele și job-ul tuturor angajaților din departamentele al căror nume conţine şirul ‘ti’. De asemenea, se va lista salariul angajaţilor, în formatul “$99,999.00”. Rezultatul se va ordona alfabetic după numele departamentului, şi în cadrul acestuia, după numele angajaţilor.

    SELECT d.department_id, department_name, job_id, last_name, to_char(salary,'$99,999.00')

    FROM employees e JOIN departments d ON (e.department_id = d.department_id)

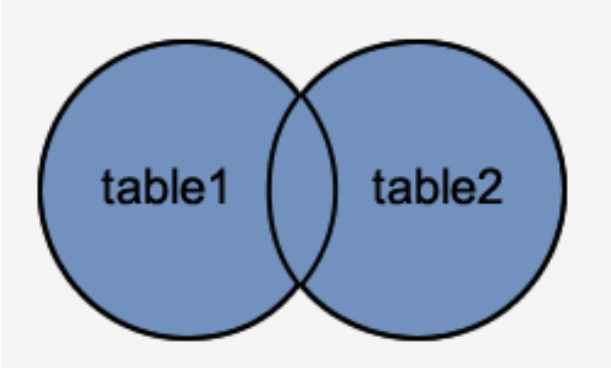
    WHERE lower(department_name) like '%ti%'

    ORDER BY _____;

CEZARA BENEGUI

12. Cum se poate implementa full outer join?

Obs: Full outer join se poate realiza fie prin reuniunea rezultatelor lui ***right outer join*** şi ***left outer join***, fie utilizând sintaxa specifică standardului SQL3.



III. [Operatori pe mulţimi]

Operatorii pe mulţimi combină rezultatele obţinute din două sau mai multe interogări. Cererile care conţin operatori pe mulţimi se numesc ***cereri compuse***.

Există patru operatori pe mulţimi: ***UNION***, ***UNION ALL***, ***INTERSECT*** şi ***MINUS***.

Toţi operatorii pe mulţimi au aceeaşi precedenţă. Dacă o instrucţiune *SQL* conţine mai mulţi operatori pe mulţimi, *server*-ul *Oracle* evaluează cererea de la stânga la dreapta (sau de sus în jos). Pentru a schimba această ordine de evaluare, se pot utiliza ***paranteze***.

- ***Operatorul UNION*** returnează toate liniile selectate de două cereri, eliminând duplicatele. Acest operator nu ignoră valorile *null* şi are precedenţă mai mică decât operatorul *IN*.

- Operatorul ***UNION ALL*** returnează toate liniile selectate de două cereri, fără a elimina duplicatele. Precizările făcute asupra operatorului *UNION* sunt valabile şi în cazul operatorului *UNION ALL*. În cererile asupra cărora se aplică *UNION ALL* nu poate fi utilizat cuvântul cheie *DISTINCT*.

- Operatorul ***INTERSECT*** returnează toate liniile comune cererilor asupra cărora se aplică. Acest operator nu ignoră valorile *null*.

- Operatorul ***MINUS*** determină liniile returnate de prima cerere care nu apar în rezultatul celei de-a doua cereri. Pentru ca operatorul *MINUS* să funcţioneze, este necesar ca toate coloanele din clauza *WHERE* să se afle şi în clauza *SELECT*.

CEZARA BENEGUI

Observaţii:

- Pentru o cerere care utilizează operatori pe mulţimi, cu excepţia lui *UNION ALL*, *server*-ul *Oracle* elimină liniile duplicat.
- În instrucţiunile *SELECT* asupra cărora se aplică operatori pe mulţimi, coloanele selectate trebuie să corespundă ca număr şi tip de date. Nu este necesar ca numele coloanelor să fie identice. Numele coloanelor din rezultat sunt determinate de numele care apar în clauza *SELECT* a primei cereri.

IV. [Exerciţii - operatori pe mulţimi]

1. Se cer **codurile departamentelor** al căror nume conţine şirul "re" sau în care lucrează angajaţi având codul job-ului "SA_REP".

   Cum este ordonat rezultatul?

2. Ce se întâmplă dacă înlocuim *UNION* cu *UNION ALL* în comanda precedentă?

3. Să se obțină **codurile departamentelor** în care nu lucrează nimeni (nu este introdus niciun salariat în tabelul *employees*). Se cer două soluții.

Obs: Operatorii pe mulţimi pot fi utilizaţi în subcereri. Coloanele care apar în clauza *WHERE* a interogării trebuie să corespundă, ca număr şi tip de date, celor din clauza *SELECT* a subcererii.

4. Se cer codurile departamentelor al căror nume conţine şirul "re" şi în care lucrează angajaţi având codul job-ului "HR_REP".

```
SELECT department_id "Cod departament"
FROM   employees
WHERE  UPPER(job_id)='HR_REP'
```

INTERSECT

```
SELECT department_id
FROM   departments
WHERE LOWER(department_name) LIKE '%re%';
```

CEZARA BENEGUI

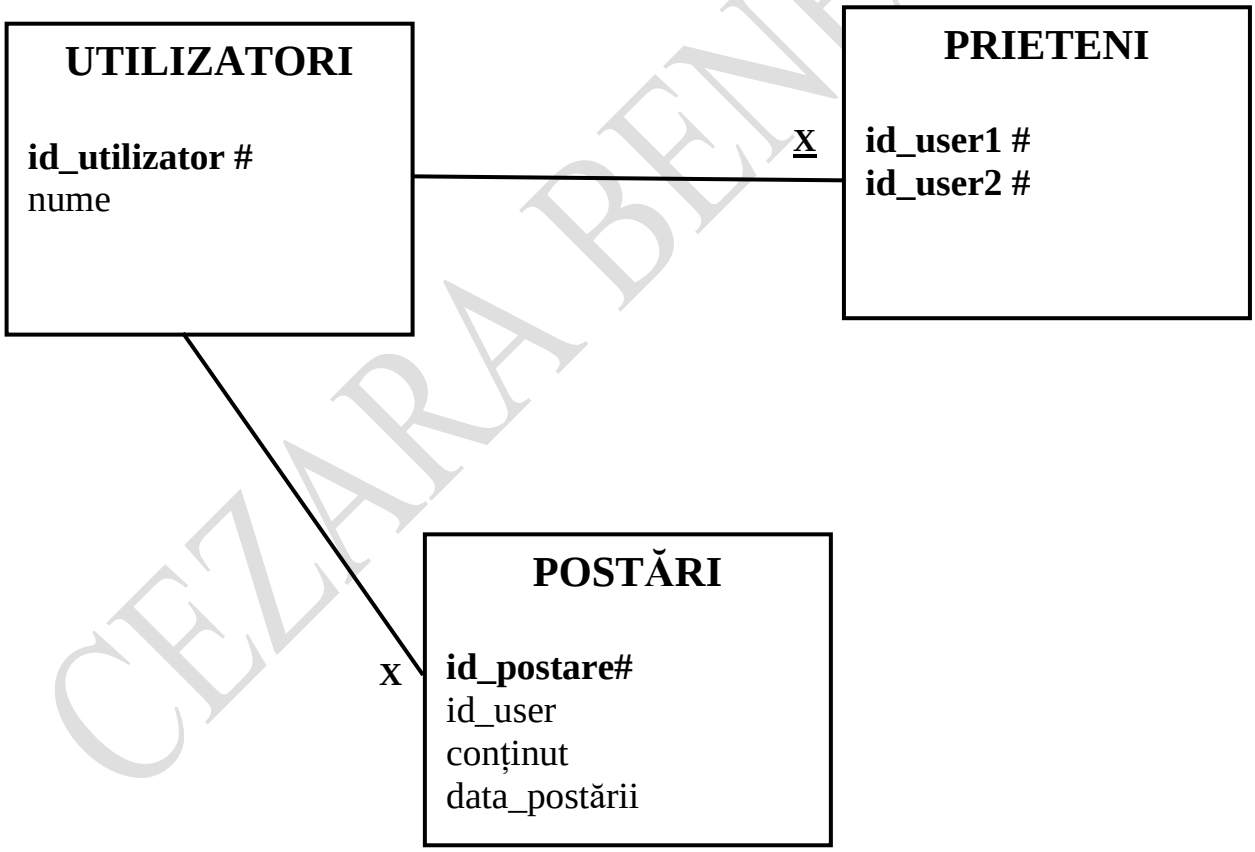
V. [Exerciţii diverse]

Mini-aplicație de tip INSTAGRAM:

OBSERVAȚIE!

Prieteniile (urmăririle) sunt **unidirecționale** – adică userul 1 îl poate urmări pe userul 2 fără ca relația să fie reciprocă.

- Un utilizator are mai mulți prieteni sau nu are prieteni;
- Un utilizator postează mai multe postări sau niciuna;



CEZARA BENEGUI

1. Afișați o listă cu toți utilizatorii și postările publicate de aceștia. Se vor afișa inclusiv utilizatorii care nu au publicat nicio postare.
   Se vor afișa codul si numele utilizatorilor, împreuna cu codul și textul postărilor. Se vor ordona postările în funcție de dată, în ordine descrescătoare, iar valorile NULL se vor afișa la finalul valorilor diferite de NULL.

2. Afișați toate postările realizate în luna **martie 2024** de către prietenii utilizatorului cu **id-ul 1**, cu excepția postărilor realizate de utilizatorii care nu îl urmăresc înapoi (adică nu există relația inversă).
   Să se afișeze codul și numele prietenului care a postat, conținutul postării și data la care a fost realizată postarea. Data o să fie formatată dd.mm.yyyy.
   Se vor eticheta toate coloanele corespunzător.

CEZARA BENEGUI